

(1) RGB

표현 방식:

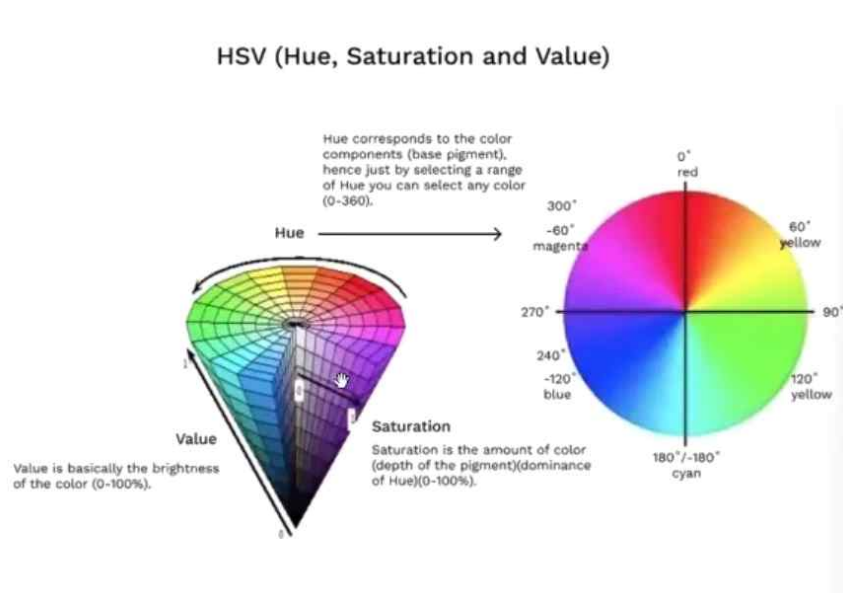
$(255, 255, 255) \rightarrow$ 흰색

$(0,0,0) \rightarrow$ 검정색

특징: 가산 혼합 방식이고 색을 더할수록 밝아지고 모두 합치면 흰색이 된다.

(2) HSV

H(Hue) 색상, S(Saturation) 채도, V(Value) 밝기



[그림1-1]

HSV 표현법은 사람 눈으로 색깔을 감지하는 방식과 더 비슷하다. [그림1-1]의 원뿔 보면 원의 각도로 색상을 표현하고 원의 중심 채도가 가장 열고 바깥쪽

으로 갈수록 짙어진다. 원뿔의 세로축으로 밝기를 표현하고 아래쪽은 어둡고 위쪽은 가장 밝다.

색상(0° - 360°): 0° =빨강, 120° =초록, 240° =파랑

채도(0% - 100%): 0% =회색 계열, 100% 순수한 색

밝기(0% - 100%): 0% =검정, 100% =최대 밝기

2. 고추 영상이 첨부되어 있다. 이 영상을 대상으로 RGB 각 성분에 대해 컬러 히스토그램을 그리시오.

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

```
# 이미지 열기
```

```
img = Image.open("gochu.jpg")
```

```
# numpy 배열 변환
```

```
img_arr = np.array(img)
```

```
# RGB 채널 분리
```

```
r = img_arr[:, :, 0]
```

```
g = img_arr[:, :, 1]
```

```
b = img_arr[:, :, 2]
```

```
# 히스토그램 그리기
```

```
plt.hist(r.flatten(), 256, color='r')
```

```
plt.hist(g.flatten(), 256, color='g')
```

```
plt.hist(b.flatten(), 256, color='b')
```

```
plt.savefig("gochucolorhistogram.png")
```

```
plt.show()
```

가로 세로 색상

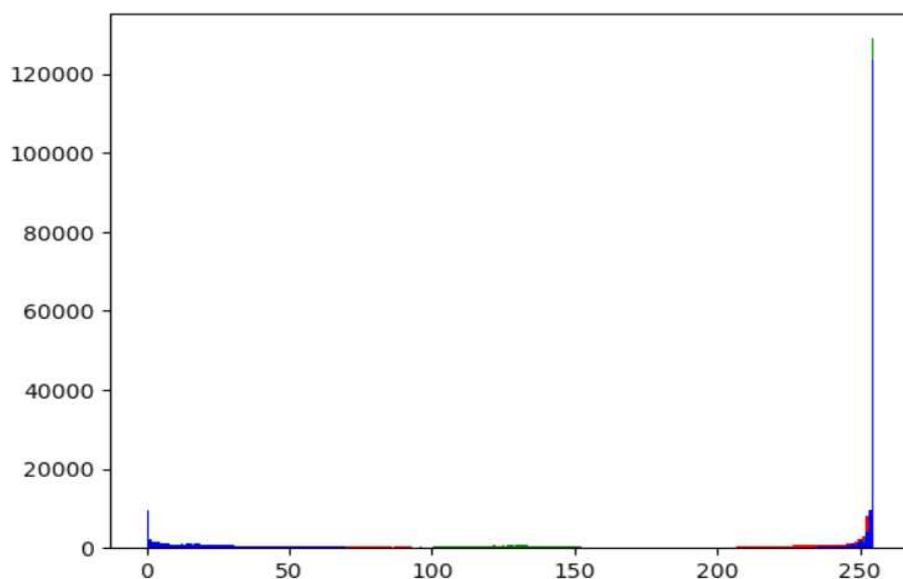
shape = (높도 H, 宽度 W, 通道 C)

3차 배열 $\begin{bmatrix} [R_1, G_1, B_1], [R_2, G_2, B_2], \dots, [R_H, G_H, B_H] \end{bmatrix}$

⋮

]

평면 배열



```

from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

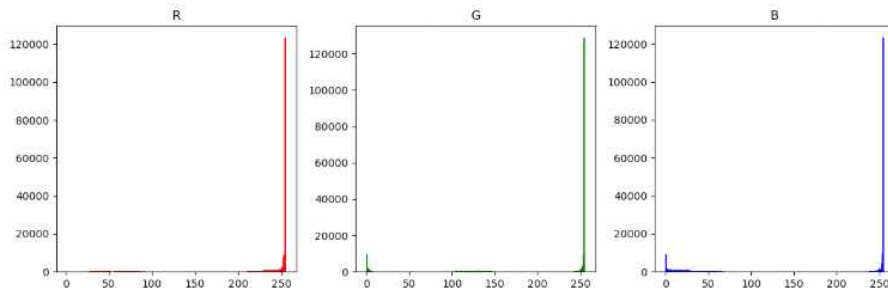
img = np.array(Image.open("gochu.jpg"))
colors = ['r', 'g', 'b'] → c
          0   1   2 → i
plt.figure(figsize=(12, 4)) → 이미지 크기 지정

for i, c in enumerate(colors):
    plt.subplot(1, 3, i+1)
    plt.hist(img[:, :, i].flatten(), 256, color=c)
    plt.title(c.upper()) → 대문자 제목

plt.tight_layout() → 자동 계산 그림 간격

plt.savefig("rgb_histogram.png") # 저장
plt.show()

```



3. 이 영상에서 붉은색 부분만 따로 추출하고 싶다. (1) RGB 표현법을 사용하여 추출해 보시오. (2) HSV 표현법을 사용하여 추출해 보시오. (3) 어느 방법이 더 나은지 눈으로 판정해 보시오.

(1) RGB 표현법

```

import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

img = Image.open("gochu.jpg").convert("RGB") →統一格式, 去掉透明度, 保证NumPy转换结果一致
img_arr = np.array(img)

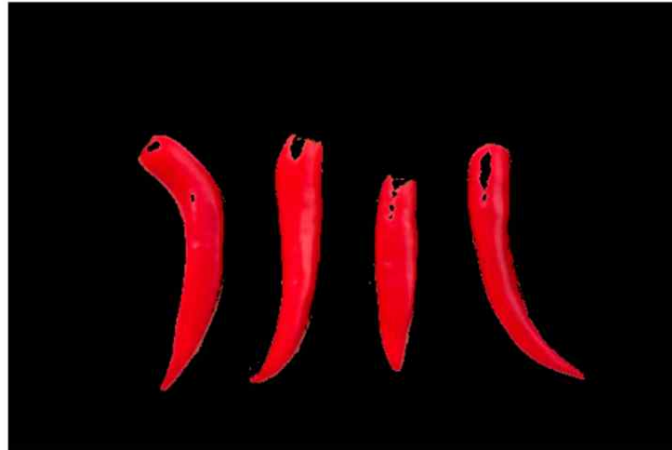
# RGB 채널 분리
r = img_arr[:, :, 0]
g = img_arr[:, :, 1]
b = img_arr[:, :, 2]

red_mask = (r > 150) & (g < 100) & (b < 100) → Boolean matrix, 元素只取0, 1.
result = img_arr.copy()
result[~red_mask] = [0, 0, 0] → 빨간색 아닌 부분은 검은색으로 바뀐다.

plt.imshow(result)
plt.title("Red channel")
plt.axis("off") → 隐藏坐标轴, 刻度边框, 只显示图像本身
plt.savefig("redRGB.png") → 저장
plt.show()

```

Red channel



(2) HSV 표현법

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from matplotlib.colors import rgb_to_hsv

img = np.array(Image.open("gochu.jpg").convert("RGB"))

# RGB -> HSV
hsv = rgb_to_hsv(img / 255.0)
H, S, V = hsv[:, :, 0], hsv[:, :, 1], hsv[:, :, 2]

mask = ((H < 0.05) | (H > 0.95)) & (S > 0.4) & (V > 0.3)

result = img.copy()
result[~mask] = [0, 0, 0]

plt.imshow(result)
plt.axis("off")
plt.title("Red Channel(HSV)")
plt.savefig("redHSV.png")
plt.show()
```

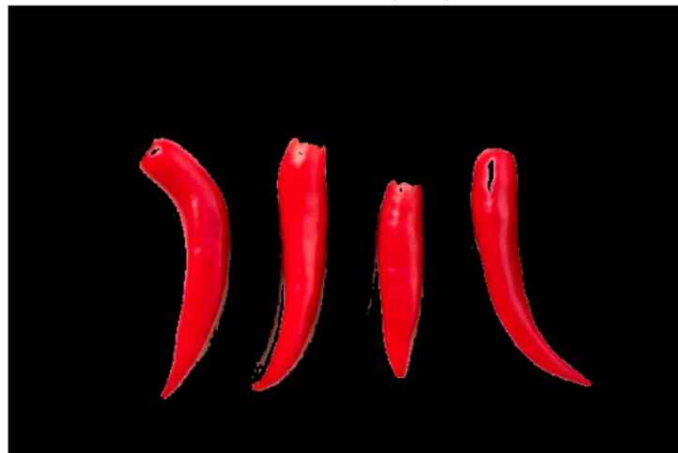
0° 또는 360° 근처.
비밀 간식.

$hsv.shape == (H, W, 3)$

$hsv[:, :, 0]$ H [0, 1]
1 S [0, 1]
2 V [0, 1]

색상.
채도
밝기

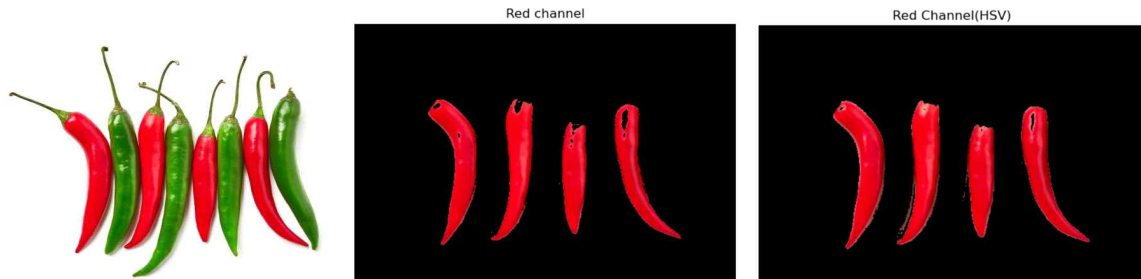
Red Channel(HSV)



(3) 비교

HSV방법은 빛의 밝기 변화에 더 예민하다는 특징을 관측될 수 있다. HSV는

다른 물체에 반사되는 붉은색 빛까지 추출되어서 원하지 않은 부분을 추출할 수 있다. RGB는 상대적으로 색깔의 밝기 변화에 예민하지 않아서 원하지 않은 부분 깔끔하게 지우지만 잃은 부분도 많다.



4. 토마토 영상이 첨부되어 있다. 이 영상을 256*256 크기의 그레이 스케일 영상으로 변환하시오.

```
from PIL import Image
import numpy as np

img= Image.open("tomato.png").convert("RGB")
img_resized=img.resize((256,256),Image.BILINEAR)

gray_img=img_resized.convert("L")
gray_img.save("gray_output.png")
```

회소근 계산하는 방식중 하나이고.

→ Bilinear Interpolation.

→ 把图像转换成单通道灰度图 (8-bit 灰度模式)

$$Y=0.299R+0.587G+0.114B$$



5. 문제 4의 그레이 스케일 영상에 대하여 (1) 3*3 크기의 소벨 오퍼레이터를 적용하여 에지를 구하시오. (2) 아래 커널을 적용하여 에지를 구해보시오. (3)

FFT를 사용하여 에지를 구하시오. (4) 어느 방법이 더 나은지 눈으로 판정해보시오.

(1) 3*3 크기의 소벨 오퍼레이터

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

$$|G| = \sqrt{G_x^2 + G_y^2}$$

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import convolve

gray_img=Image.open("gray_output.png").convert("L")
gray=np.array(gray_img, dtype=np.float32)
#Sobel 卷积核
sobel_x = np.array([[ -1,  0,  1],
                    [ -2,  0,  2],
                    [ -1,  0,  1]], dtype=np.float32)

sobel_y = np.array([[ -1, -2, -1],
                    [  0,  0,  0],
                    [  1,  2,  1]], dtype=np.float32)

#卷积计算
gx = convolve(gray, sobel_x)
gy = convolve(gray, sobel_y)
#得到边缘值
sobel_mag = np.sqrt(gx**2 + gy**2)
sobel_mag = np.clip(sobel_mag, 0, 255).astype(np.uint8)
Image.fromarray(sobel_mag).save("edge_sobel.png")
```

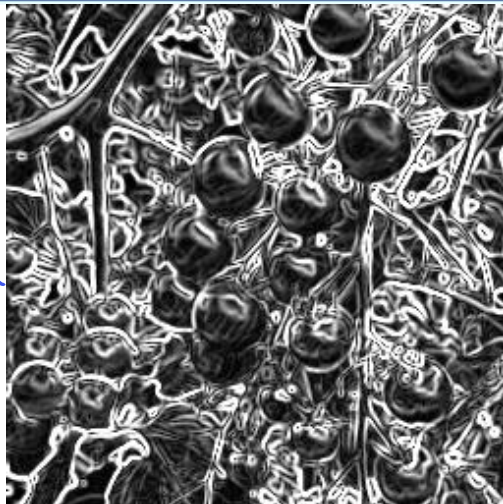
对数组做
“卷积运算”
滤波
 $y = \sum_i x_i \cdot k_i$
 $k =$ 卷积核

→ ≥ 0
把数组中的元素限制在一个最小值和最大值之间。

Pillow(PIL)里
把Numpy数组
转回图像2维数
的函数。

(dtype必须是
uint8)

(2) 주어진 커널 적용



0.5	1.5	0.5
1.5	-8	1.5
0.5	1.5	0.5

```

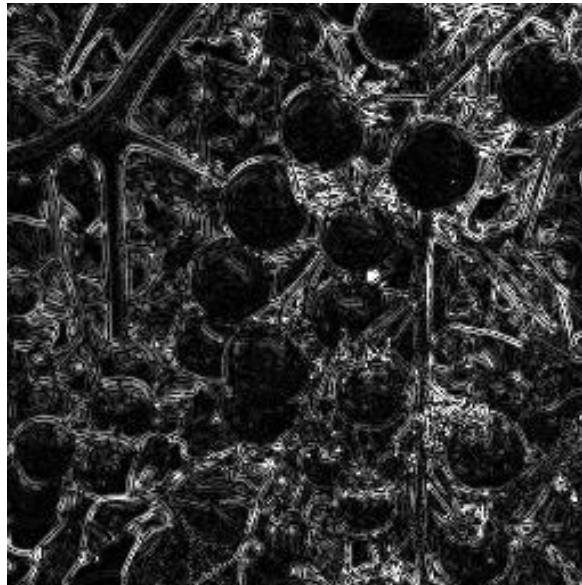
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import convolve

kernel = np.array([[ 0.5, 1.5, 0.5],
                    [1.5, -8, 1.5],
                    [0.5, 1.5, 0.5]], dtype=np.float32)

edge_lap = convolve(gray, kernel)
#取绝对值并归一化
edge_lap = np.abs(edge_lap)
edge_lap = np.clip(edge_lap, 0, 255).astype(np.uint8)

Image.fromarray(edge_lap).save("edge_kernel.png")

```



공간영역

주파수영역

(3) FFT

图像 → 傅里叶变换 →
 → 频域
 高频分量 ≈ 边缘
 设计高通滤波器
 逆FFT回到空间域

from numpy.fft import fft, fftshift.

```

# FFT
fft_img = np.fft.fft2(gray) → 对二维数据进行二维快速傅里叶变换 (2DFFT)
fft_shift = np.fft.fftshift(fft_img) → 把图像从“空间域”变换到“频域”
h, w = gray.shape
cx, cy = h // 2, w // 2 → 找到频谱中心
mask = np.ones((h, w), np.uint8)
r = 30 # 截止频率
y, x = np.ogrid[:h, :w]
mask[(x - cx)**2 + (y - cy)**2 < r*r] = 0 → 阻挡圆内低频, 让高频通过
fft_filtered = fft_shift * mask
# 逆FFT
ifft = np.fft.ifftshift(fft_filtered) → 还原顺序
edge_fft = np.fft.ifft2(ifft) → 取出结果
edge_fft = np.abs(edge_fft)
# 归一化
edge_fft = np.clip(edge_fft, 0, 255).astype(np.uint8)

Image.fromarray(edge_fft).save("edge_fft.png")

```

把FFT结果的零频移到频谱中心。

np.ogrid → 生成“行向量 + 列向量”

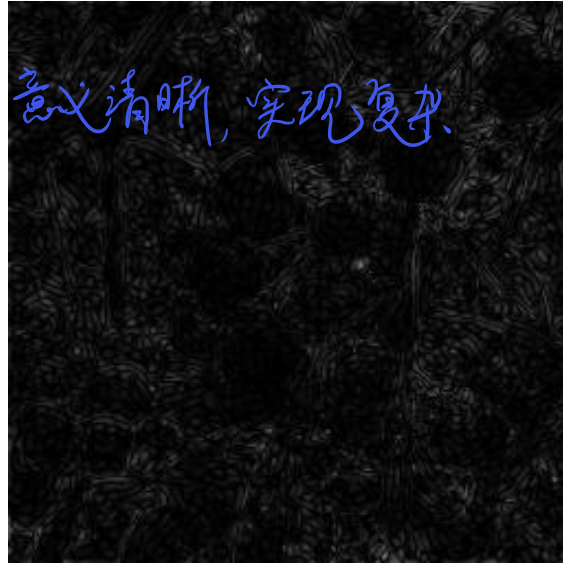
优点

缺点

Sobel: 空间域, 稳定易实现, 边缘较粗.

Kernel: 空间域, 定位准, 噪声敏感

FFT: 频域, 物理意义清晰, 实现复杂.



(4) 비교

소벨 오퍼레이터를 사용했을 때 에지 제일 뚜렷하다.

6. (1) 문제 4의 그레이 스케일 영상에 대하여 5*5 크기에 sigma=1.0인 가우시안 필터를 적용하여 스무딩하시오. (2) 그 스무딩한 영상을 대상으로 5*5 크기의 라플라시안 오퍼레이터를 적용하여 에지를 구하시오. (3) 원래의 그레이 스케일 영상에서 에지 부분은 값을 255로 하고, 나머지는 그대로 두시오.

(1) 가우시안 스무딩

高斯滤波:

抑制噪声

保留低频结构

使边缘更稳定.

数学表达:

1. 高斯滤波

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

2. 拉普拉斯算子.

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

3. LoG: $LoG(x,y) = \nabla^2 (G * I)$

```
from PIL import Image
import numpy as np
from scipy.ndimage import gaussian_filter

gray_img=Image.open("gray_output.png").convert("L")
gray=np.array(gray_img, dtype=np.float32)

smoothed = gaussian_filter(gray,sigma=1.0)

smoothed_img= Image.fromarray(smoothed.astype(np.uint8))
smoothed_img.save("gray_smoothed_gaussian.png")
```

→ 越大越模糊



(2) 라플라시안 에지

```

from PIL import Image
import numpy as np
from scipy.ndimage import gaussian_filter, convolve

gray_img=Image.open("gray_output.png").convert("L")
gray=np.array(gray_img, dtype=np.float32)

smoothed = gaussian_filter(gray,sigma=1.0)

laplacian_operater=np.array([
    [0, 0, -1, 0, 0],
    [0, -1,-2,-1, 0],
    [-1,-2,16,-2,-1],
    [0, -1,-2,-1, 0],
    [0, 0, -1, 0, 0]],dtype=np.float32)

lap_edge = convolve(smoothed,laplacian_operater)
lap_edge=np.abs(lap_edge)
lap_edge= np.clip(lap_edge, 0, 255).astype(np.uint8)

edge_img=Image.fromarray(lap_edge)
edge_img.save("laplacian_edge.png")

```

拉普拉斯算子.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

并在平滑图像上应用拉普拉斯

lap_edge = convolve(smoothed,laplacian_operater)

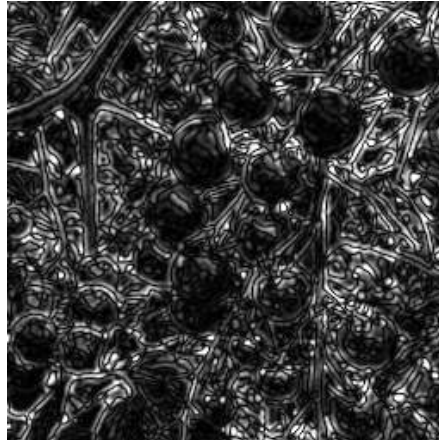
| #取绝对值并归一化

lap_edge=np.abs(lap_edge)

lap_edge= np.clip(lap_edge, 0, 255).astype(np.uint8)

edge_img=Image.fromarray(lap_edge)

edge_img.save("laplacian_edge.png")



(3) 에지 부분 255로 변환

에지
구하기

```
from PIL import Image
import numpy as np
from scipy.ndimage import gaussian_filter, convolve

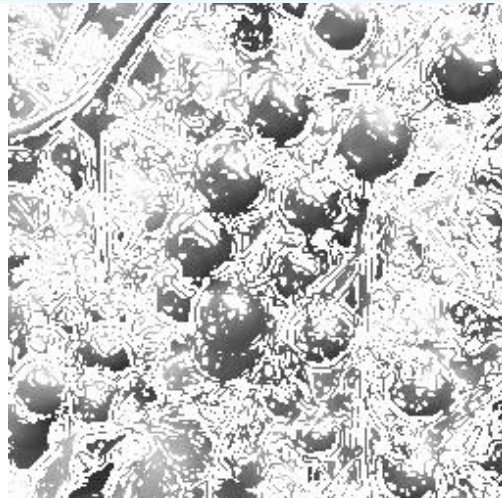
gray_img=Image.open("gray_output.png").convert("L")
gray=np.array(gray_img, dtype=np.float32)

smoothed = gaussian_filter(gray,sigma=1.0)

laplacian_operator=np.array([
    [0, 0, -1, 0, 0],
    [0, -1,-2,-1, 0],
    [-1,-2,16,-2,-1],
    [0, -1,-2,-1, 0],
    [0, 0, -1, 0, 0]],dtype=np.float32)

lap_edge = convolve(smoothed,laplacian_operator)

lap_edge=np.abs(lap_edge)
#简单阈值化,得到二值边缘.
threshold=30
edge_mask=(lap_edge> threshold).astype(np.uint8)
#참이언!
result=gray.copy()
result[edge_mask == 1]=255
result_img= Image.fromarray(result.astype(np.uint8)).save("gray_with_laplacian_edge.png")
```



※ 이상의 문제들에서 처리한 영상은 모두 디스플레이하고 또 저장(파일명은
알아서 정할 것)하시오.

※ 소스코드는 단순명료할수록 좋고, numpy, matplotlib, pillow, scipy,
skimage, openCV 라이브러리를 이용하여 해결하시오. 이외의 다른 라이브러
리는 가급적 사용하지 마시오.

[그림1-1] 출처:

[https://medium.com/@dijdomv01/a-beginners-guide-to-understand-th
e-color-models-rgb-and-hsv-244226e4b3e3](https://medium.com/@dijdomv01/a-beginners-guide-to-understand-the-color-models-rgb-and-hsv-244226e4b3e3)